



Logo UASLP

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ

FACULTAD DE ESTUDIOS
PROFESIONALES
ZONA MEDIA

Ingeniería en Mecatrónica



FACULTAD DE
ESTUDIOS
PROFESIONALES
ZONA MEDIA
UASLP

Logo FEPZM

TAREA

EJERCICIOS 67–80

Interpolación y Ajuste de Curvas

Método de Mínimos Cuadrados

Capítulo 3

Materia: Matemáticas por Computadora

Docente: Ing. Jesús Padrón

Alumno: Carlos Ángel García Sifuentes

Carrera: Ingeniería en Mecatrónica

Semestre: 4° Semestre

Fecha: 18 de mayo de 2026

Ejercicios 67–80: Mínimos Cuadrados e Interpolación

Implementación en Python

CARLOS ÁNGEL GARCÍA SIFUENTES
a383917@alumnos.uaslp.mx

Docente: Ing. Jesús Padrón
Fecha: 18 de mayo de 2026

Resumen—Se resuelven los ejercicios 67 al 80 del capítulo de Interpolación y Ajuste de Curvas, correspondientes a la técnica de mínimos cuadrados. Para cada ejercicio se presenta el desarrollo matemático completo, las ecuaciones normales y la implementación computacional en Python con NumPy. Se abordan ajuste lineal, polinomial, ajuste de planos, modelos exponenciales y potenciales, y el análisis del condicionamiento numérico de la matriz de diseño.

I. EJERCICIO 67 — DEMOSTRACIÓN: $(\bar{X}, \bar{Y})$ EN LA RECTA

Enunciado: Mostrar que $(\bar{X}, \bar{Y})$ es siempre un punto sobre la recta de mínimos cuadrados para cualquier conjunto de datos.

I-A Desarrollo matemático

La recta de mínimos cuadrados tiene la forma:

$$y = a + bx \quad (1)$$

Minimizando $S = \sum_{i=1}^n (y_i - a - bx_i)^2$ e igualando las derivadas parciales a cero, se obtienen las **ecuaciones normales**:

$$na + b \sum x_i = \sum y_i \quad (2)$$

$$a \sum x_i + b \sum x_i^2 = \sum x_i y_i \quad (3)$$

Dividiendo (2) entre n :

$$a + b \bar{X} = \bar{Y} \quad (4)$$

Esto prueba que el centroide $(\bar{X}, \bar{Y})$ satisface exactamente la ecuación de la recta, independientemente de los datos. ■

CÓDIGO PYTHON – EJERCICIO 67

Verificación numérica

```
import numpy as np

x = np.array([1.2, 3.1, 5.6, 6.2, 8.8, 9.1])
y = np.array([4.957, 12.909, 23.404, 25.981, 36.907, 38.212])
n = len(x)

Xbar, Ybar = np.mean(x), np.mean(y)
b = (n*np.sum(x*y) - np.sum(x)*np.sum(y)) / \
    (n*np.sum(x**2) - np.sum(x)**2)
a = (np.sum(y) - b*np.sum(x)) / n

y_en_Xbar = a + b * Xbar
print(f"y(X_bar) = {y_en_Xbar:.6f}")
print(f"Y_bar = {Ybar:.6f}")
print(f"Son iguales: {np.isclose(y_en_Xbar, Ybar)}") # True
```

II. EJERCICIO 68 — AJUSTE LINEAL BIDIRECCIONAL

Datos:

x	1.2	3.1	5.6	6.2	8.8	9.1
y	4.957	12.909	23.404	25.981	36.907	38.212

II-A Parte a) — x libre de errores: $y = f(x)$

Se minimizan las desviaciones **verticales**. Sumas requeridas:

$$\begin{aligned} \sum x_i &= 34.0, \quad \sum y_i = 142.37, \quad \sum x_i^2 = 241.10, \quad \sum x_i y_i = 1010.622 \\ b &= \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} = \frac{6(1010.622) - (34)(142.37)}{6(241.10) - (34)^2} = \frac{1223.15}{290.6} \approx \end{aligned} \quad (5)$$

$$a = \frac{\sum y_i - b \sum x_i}{n} = \frac{142.37 - 4.209(34)}{6} \approx -0.123 \quad (6)$$

$$\boxed{y = -0.123 + 4.209x} \quad (7)$$

II-B Parte b) – y libre de errores: $x = g(y)$

Se minimizan desviaciones **horizontales**:

$$d = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum y_i^2 - (\sum y_i)^2} \approx 0.2376, \quad c \approx 0.029 \quad (8)$$

Reescrita como $y = f(x)$: $y \approx -0.122 + 4.209x$

II-C Partes c) y d) – Comparación

Las rectas son prácticamente idénticas ($R^2 \approx 1$). La suma de cuadrados SS es siempre menor en la dirección que cada ajuste minimiza.

CÓDIGO PYTHON – EJERCICIO 68

Ajuste lineal bidireccional

```
import numpy as np

x = np.array([1.2, 3.1, 5.6, 6.2, 8.8, 9.1])
y = np.array([4.957, 12.909, 23.404, 25.981, 36.907, 38.212])
n = len(x)

# Parte a) x libre de errores
b_a = (n*np.sum(x*y) - np.sum(x)*np.sum(y)) / \
      (n*np.sum(x**2) - np.sum(x)**2)
a_a = (np.sum(y) - b_a*np.sum(x)) / n
SS_a = np.sum((y - (a_a + b_a*x))**2)
print(f"a) y = {a_a:.4f} + {b_a:.4f}*x    SS = {SS_a:.6f}")

# Parte b) y libre de errores
bb = (n*np.sum(x*y) - np.sum(x)*np.sum(y)) / \
     (n*np.sum(y**2) - np.sum(y)**2)
a_b = (np.sum(x) - bb*np.sum(y)) / n
slope = 1.0 / bb
intercept = -a_b / bb
SS_b = np.sum((x - (a_b + bb*y))**2)
print(f"b) y = {intercept:.4f} + {slope:.4f}*x    SS={SS_b:.6f}")
```

III. EJERCICIO 69 – AJUSTE DE PLANO $Z = AX + BY + C$

Datos:

x	0.40	1.2	3.4	4.1	5.7	7.2	9.3
y	0.70	2.1	4.0	6.3	6.3	8.1	8.9
z	0.031	0.933	3.058	3.349	4.870	5.757	8.921

III-A Parte a) – Ecuaciones normales

Minimizando $S = \sum (z_i - ax_i - by_i - c)^2$:

$$\begin{bmatrix} \sum x_i^2 & \sum x_i y_i & \sum x_i \\ \sum x_i y_i & \sum y_i^2 & \sum y_i \\ \sum x_i & \sum y_i & n \end{bmatrix} \begin{bmatrix} a \\ b \\ c \end{bmatrix} = \begin{bmatrix} \sum x_i z_i \\ \sum y_i z_i \\ \sum z_i \end{bmatrix} \quad (9)$$

III-B Partes b) y c) – Solución matricial

Usando $A^T A c = A^T z$ con $A = [x, y, 1]$, se obtiene el plano ajustado y la suma de cuadrados de las desviaciones.

CÓDIGO PYTHON – EJERCICIO 69

Ajuste de plano

```
import numpy as np

x = np.array([0.40, 1.2, 3.4, 4.1, 5.7, 7.2, 9.3])
y = np.array([0.70, 2.1, 4.0, 6.3, 6.3, 8.1, 8.9])
z = np.array([0.031, 0.933, 3.058, 3.349, 4.870, 5.757, 8.921])

A = np.column_stack([x, y, np.ones(len(x))])
AtA = A.T @ A
Atz = A.T @ z

coef = np.linalg.solve(AtA, Atz)
a_c, b_c, c_c = coef
print(f"z = {a_c:.4f}*x + {b_c:.4f}*y + {c_c:.4f}")

z_hat = A @ coef
SS = np.sum((z - z_hat)**2)
print(f"Suma de cuadrados SS = {SS:.6f}")
```

IV. EJERCICIO 70 – MÍNIMOS CUADRADOS CON TERCER PUNTO EN $X = 4$

La recta original entre (2, 5) y (6, -1) tiene pendiente:

$$m = \frac{-1 - 5}{6 - 2} = -\frac{3}{2} \implies y = 8 - 1.5x \quad (10)$$

Al agregar el punto (4, y_3) y calcular la recta de mini-

mos cuadrados:

$$b = \frac{3 \sum x_i y_i - \sum x_i \sum y_i}{3 \sum x_i^2 - (\sum x_i)^2} = \frac{-36}{24} = -1.5 \quad (\text{independiente}) \quad (11)$$

La pendiente siempre es -1.5 . Solo el intercepto cambia:

$$a = \frac{22 + y_3}{3} \quad (12)$$

CÓDIGO PYTHON – EJERCICIO 70

Efecto del tercer punto

```
import numpy as np

def ls_line(xs, ys):
    n = len(xs)
    b = (n*np.sum(xs*ys) - np.sum(xs)*np.sum(ys)) / \
        (n*np.sum(xs**2) - np.sum(xs)**2)
    a = (np.sum(ys) - b*np.sum(xs)) / n
    return a, b

casos = [("a" y3=5", 5.0), ("b" y3=0", 0.0),
         ("c" y3=4", 4.0), ("d" y3=-1", -1.0)]

for label, y3 in casos:
    xs = np.array([2., 4., 6.])
    ys = np.array([5., y3, -1.])
    a, b = ls_line(xs, ys)
    SS = np.sum((ys - (a + b*xs))**2)
    print(f"{label}: y = {a:.4f} + {b:.4f}*x SS={SS:.4f}")
```

V. EJERCICIO 71 – AJUSTE EXPONENCIAL

$$S = AE^{BT}$$

Datos de solubilidad de n-butano:

T (°F)	77	100	185	239	285
S (wt %)	2.4	3.4	7.0	11.1	19.6

Usando $Y = \ln S$ y $X = T$, el modelo se vuelve lineal:

$$Y = \ln A + BX \quad (13)$$

Con los datos T y S se obtiene:

- $B \approx 0.00506$
- $\ln A \approx 0.5259 \Rightarrow A \approx 1.691$

Modelo ajustado:

$$S \approx 1.691 e^{0.00506 T} \quad (14)$$

CÓDIGO PYTHON – EJERCICIO 71

Ajuste exponencial

```
import numpy as np

T = np.array([77., 100., 185., 239., 285.])
S = np.array([2.4, 3.4, 7.0, 11.1, 19.6])
n = len(T)
Y = np.log(S)

B = (n*np.sum(T*Y) - np.sum(T)*np.sum(Y)) / \
    (n*np.sum(T**2) - np.sum(T)**2)
lnA = (np.sum(Y) - B*np.sum(T)) / n
A = np.exp(lnA)
print(f"S = {A:.3f} * exp({B:.5f} * T)")
```

VI. EJERCICIO 72 – ANÁLISIS GRÁFICO: PAPEL ORDINARIO VS SEMI-LOGARÍTMICO

VI-A Partes a) y b)

En papel ordinario (T vs S) los puntos no forman una línea recta. En papel semi-logarítmico (T vs $\ln S$) los puntos caen aproximadamente sobre una recta, validando el modelo $S = Ae^{BT}$.

VI-B Parte c) – Confirmación

La ecuación $\ln S = \ln A + BT$ es algebraicamente equivalente a $S = Ae^{BT}$. El coeficiente R^2 en escala semi-log supera al de escala lineal.

CÓDIGO PYTHON – EJERCICIO 72

Comparación R^2 : lineal vs semi-logarítmico

```
import numpy as np

T = np.array([77., 100., 185., 239., 285.])
S = np.array([2.4, 3.4, 7.0, 11.1, 19.6])
lnS = np.log(S)

r2_lin = np.corrcoef(T, S)[0, 1]**2
r2_log = np.corrcoef(T, lnS)[0, 1]**2
print(f"R^2 lineal: {r2_lin:.4f}")
print(f"R^2 semi-log: {r2_log:.4f} <--- mejor ajuste")
```

VII. EJERCICIO 73 – COMPARACIÓN: LINEAL, CUADRÁTICO Y CÚBICO

Se generan 6 valores $x_i \in [2, 7]$ aleatorios con ruido $z_i \in [0, 0.2]$ y $y_i = x_i^2 + z_i$.

VII-A Ajuste polinomial de grado k

Para cada grado k se construye $A = [\mathbf{1}, \mathbf{x}, \dots, \mathbf{x}^k]$ y se resuelve $A^T A \mathbf{c} = A^T \mathbf{y}$. El modelo cuadrático es el óptimo porque los datos son exactamente cuadráticos más ruido.

CÓDIGO PYTHON – EJERCICIO 73

Comparación de grados de ajuste

```
import numpy as np

np.random.seed(42)
x = np.sort(np.random.uniform(2, 7, 6))
z = np.random.uniform(0, 0.2, 6)
y = x**2 + z

for deg in [1, 2, 3]:
    A = np.column_stack([x**i for i in range(deg + 1)])
    coef = np.linalg.lstsq(A, y, rcond=None)[0]
    SS = np.sum((y - A @ coef)**2)
    print(f"Grado {deg}: SS = {SS:.8f}")
```

VIII. EJERCICIO 74 – MATRIZ DE DISEÑO A Y ECUACIONES NORMALES

VIII-A Parte a) – $A^T A$ produce la matriz de coeficientes

Para el ajuste $P_n(x) = c_0 + c_1 x + \dots + c_n x^n$:

$$A = \begin{bmatrix} 1 & x_1 & \dots & x_1^n \\ 1 & x_2 & \dots & x_2^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & \dots & x_m^n \end{bmatrix} \quad (15)$$

El problema minimiza $S = \|A\mathbf{c} - \mathbf{y}\|^2$, cuyas ecuaciones normales son $A^T A \mathbf{c} = A^T \mathbf{y}$. Al expandir:

$$A^T A = \begin{bmatrix} m & \sum x_i & \dots & \sum x_i^n \\ \sum x_i & \sum x_i^2 & \dots & \sum x_i^{n+1} \\ \vdots & \vdots & \ddots & \vdots \\ \sum x_i^n & \sum x_i^{n+1} & \dots & \sum x_i^{2n} \end{bmatrix} \quad (16)$$

que coincide con la matriz de la Ec. (3.28). ■

VIII-B Parte b) – $A^T \mathbf{y}$ produce el lado derecho

$$A^T \mathbf{y} = \left(\sum y_i, \sum x_i y_i, \dots, \sum x_i^n y_i \right)^T \quad \blacksquare \quad (17)$$

VIII-C Parte c) – Mayor economía con A directa

`numpy.linalg.lstsq(A, y)` usa factorización QR/SVD internamente, más estable que construir $A^T A$ y resolverlo directamente, especialmente para grados altos donde las potencias x^{2k} causan pérdida de precisión.

CÓDIGO PYTHON – EJERCICIO 74

Verificación matricial

```
import numpy as np

x = np.array([1.2, 3.1, 5.6, 6.2, 8.8, 9.1])
y = np.array([4.957, 12.909, 23.404, 25.981, 36.907, 38.212])
n = len(x)

A = np.column_stack([np.ones(n), x])
AtA = A.T @ A
Aty = A.T @ y

print("A^T A ="); print(AtA)
print("\nA^T y ="); print(Aty)

coef = np.linalg.lstsq(A, y, rcond=None)[0]
print(f"\na = {coef[0]:.4f}, b = {coef[1]:.4f}")
```

IX. EJERCICIO 75 – AJUSTE POTENCIAL $F = KP^n$

Datos:

Flujo (F)	94	118	147	180	230
Presión (P)	10	16	25	40	60

Linealizando con $\ln F = \ln k + n \ln P$:

$$n = \frac{N \sum \ln P_i \ln F_i - \sum \ln P_i \sum \ln F_i}{N \sum (\ln P_i)^2 - (\sum \ln P_i)^2} \approx 0.476 \quad (18)$$

$$\ln k = \frac{\sum \ln F_i - n \sum \ln P_i}{N} \Rightarrow k \approx 38.7 \Rightarrow F = 38.7 P^{0.476} \quad (19)$$

El exponente ≈ 0.5 confirma que $F \propto \sqrt{P}$.

CÓDIGO PYTHON – EJERCICIO 75

Ajuste potencial en escala log-log

```
import numpy as np

P = np.array([10., 16., 25., 40., 60.])
F = np.array([94., 118., 147., 180., 230.])
N = len(P)
lnP, lnF = np.log(P), np.log(F)

n_exp = (N*np.sum(lnP*lnF) - np.sum(lnP)*np
        .sum(lnF)) / \
        (N*np.sum(lnP**2) - np.sum(lnP)**2)
lnk = (np.sum(lnF) - n_exp*np.sum(lnP)) / N
k = np.exp(lnk)
print(f"F = {k:.4f} * P^{n_exp:.4f}")

r2 = np.corrcoef(lnP, lnF)[0, 1]**2
print(f"R^2 en log-log: {r2:.6f}")
```

X. EJERCICIO 76 – AJUSTE CUADRÁTICO

$$F = AP^2 + BP + C$$

X-A Parte a) – Coeficientes por mínimos cuadrados

Con $A = [P^2, P, 1]$ se resuelve $A^T A c = A^T F$.

X-B Parte b) – Comparación con ejercicio 75

La SS del modelo cuadrático puede ser menor en valor absoluto, pero el modelo potencial es físicamente más apropiado y más parsimonioso (2 parámetros vs 3).

CÓDIGO PYTHON – EJERCICIO 76

Ajuste cuadrático vs potencial

```
import numpy as np

P = np.array([10., 16., 25., 40., 60.])
F = np.array([94., 118., 147., 180., 230.])

A = np.column_stack([P**2, P, np.ones(
    len(P))])
coef = np.linalg.lstsq(A, F, rcond=None)[0]
a, b, c = coef
F_hat = A @ coef
SS_quad = np.sum((F - F_hat)**2)
print(f"F = {a:.6f}*P^2 + {b:.4f}*P + {c:.4f}")
print(f"SS cuadrático = {SS_quad:.4f}")

k, n_exp = 38.66, 0.4764
SS_pot = np.sum((F - k * P**n_exp)**2)
```

```
print(f"SS potencial = {SS_pot:.4f}")
```

XI. EJERCICIO 77 – POLINOMIO DE GRADO ÓPTIMO

Datos:

x	1.1	1.6	11.4	4.1	5.3	17.5	9.4	11.5
$f(x)$	7.9	24.8	-28.8	42.6	29.6	-34.6	-3.1	-28.7

XI-A Criterio de selección

Para cada grado n se minimiza $SS = \sum (f_i - P_n(x_i))^2$. El grado óptimo es donde la reducción en SS se vuelve despreciable. Si el grado es excesivo aparece **sobreajuste** y pérdida de estabilidad numérica.

CÓDIGO PYTHON – EJERCICIO 77

Selección de grado óptimo

```
import numpy as np

x = np.array([1.1, 1.6, 11.4, 4.1, 5.3,
              17.5, 9.4, 11.5, 12.1])
fx = np.array([7.9, 24.8, -28.8, 42.6,
               29.6, -34.6, -3.1, -28.7, -39.6])

prev_ss = None
for deg in range(1, 8):
    A = np.column_stack([x**i for i in
                          range(deg + 1)])
    coef = np.linalg.lstsq(A, fx, rcond=
                           None)[0]
    SS = np.sum((fx - A @ coef)**2)
    delta = f"{prev_ss - SS:.6f}" if
    prev_ss else ""
    print(f"Grado {deg}: SS = {SS:.6f}
          Mejora = {delta}")
    prev_ss = SS
```

Un polinomio de grado 4 o 5 representa el mejor balance entre precisión y simplicidad.

XII. EJERCICIO 78 – REPETICIÓN CON SUB-CONJUNTOS DE PUNTOS

XII-A Parte a) – Puntos en índices pares (0, 2, 4, 6, 8)

Cinco puntos; el grado óptimo puede reducirse.

XII-B Parte b) – Puntos en índices impares (1, 3, 5, 7)

Cuatro puntos; aún menos restricciones.

XII-C Parte c) – Comparación con ejercicio 77

Los subconjuntos producen coeficientes distintos. Usar todos los datos da mayor confiabilidad estadística.

CÓDIGO PYTHON – EJERCICIO 78

Ajuste por subconjuntos de puntos

```
import numpy as np

x = np.array([1.1, 1.6, 11.4, 4.1, 5.3,
              17.5, 9.4, 11.5, 12.1])
fx = np.array([7.9, 24.8, -28.8, 42.6,
               29.6, -34.6, -3.1, -28.7, -39.6])

def poly_ss(xs, ys, max_deg=5):
    for deg in range(1, min(len(xs),
                             max_deg + 1)):
        A = np.column_stack([xs**i for i
                              in range(deg + 1)])
        coef = np.linalg.lstsq(A, ys, rcond
                                =None)[0]
        SS = np.sum((ys - A @ coef)**2)
        print(f"Grado {deg}: SS = {SS:.4f}
              ")

print("Parte a) Índices pares:")
poly_ss(x[::2], fx[::2])
print("Parte b) Índices impares:")
poly_ss(x[1::2], fx[1::2])
```

XIII-C Parte c) – Cálculo de a y b

$$b = \frac{n \sum u_i f_i - \sum u_i \sum f_i}{n \sum u_i^2 - (\sum u_i)^2}, \quad a = \frac{\sum f_i - b \sum u_i}{n} \quad (20)$$

CÓDIGO PYTHON – EJERCICIO 79

Ajuste seno con c conocido

```
import numpy as np

x = np.array([1.1, 1.6, 11.4, 4.1, 5.3,
              17.5, 9.4, 11.5, 12.1])
fx = np.array([7.9, 24.8, -28.8, 42.6,
               29.6, -34.6, -3.1, -28.7, -39.6])

c = np.pi / 10
u = np.sin(c * x)
n = len(x)

b = (n*np.sum(u*fx) - np.sum(u)*np.sum(fx)) / \
    (n*np.sum(u**2) - np.sum(u)**2)
a = (np.sum(fx) - b*np.sum(u)) / n
SS = np.sum((fx - (a + b*u))**2)
print(f"a = {a:.4f}, b = {b:.4f}")
print(f"f(x) = {a:.4f} + {b:.4f} * sin(pi
      /10 * x)")
print(f"SS = {SS:.4f}")
```

XIII. EJERCICIO 79 – AJUSTE $F(X) = A + B \sin(CX)$

XIII-A Parte a) – Dificultad con c desconocido

Si c es desconocido el modelo es **no lineal** en los parámetros. Las ecuaciones normales forman un sistema no lineal que no puede resolverse directamente; se requiere minimización iterativa (p.ej. Levenberg-Marquardt).

XIII-B Parte b) – Con $c = \pi/10$ conocido

Definiendo $u_i = \sin\left(\frac{\pi}{10} x_i\right)$, el modelo se vuelve lineal: $f(x) = a + bu$ (regresión lineal estándar en (u_i, f_i)).

XIV. EJERCICIO 80 – CONDICIONAMIENTO DE LA MATRIZ PARA $P_N(X)$

Con 10 puntos igualmente espaciados en $[3, 7]$, el número de condición $\kappa(A^T A)$ crece exponencialmente con el grado:

Grado	$\kappa(A^T A)$ aprox.	$\log_{10}(\kappa)$
P_4	$\sim 10^8$	~ 8
P_6	$\sim 10^{14}$	~ 14
P_8	$\sim 10^{18}$	~ 18

Esto confirma la advertencia de la Sección 3.6: para grados mayores a 3 o 4 la matriz es numéricamente inestable. Se recomiendan polinomios ortogonales o el uso directo de `numpy.linalg.lstsq` (QR/SVD).

CÓDIGO PYTHON – EJERCICIO 80

Número de condición por grado

```
import numpy as np

x = np.linspace(3, 7, 10)
print(f"{'Grado':>7} {'Cond(AtA)':>16} {'log10(cond)':>12}")
for deg in [4, 6, 8]:
    A = np.column_stack([x**i for i in range(deg + 1)])
    AtA = A.T @ A
    cond = np.linalg.cond(AtA)
    print(f" P_{deg}(x): {cond:>16.2e} {np.log10(cond):>12.2f}")

# Comparar estabilidad: solve vs lstsq
deg = 8
A = np.column_stack([x**i for i in range(deg + 1)])
y_test = x**3 - 2*x + 1
try:
    c_s = np.linalg.solve(A.T @ A, A.T @ y_test)
    print(f"SS solve: {np.sum((y_test - A@c_s)**2):.4e}")
except np.linalg.LinAlgError:
    print("solve: matriz singular")
c_l = np.linalg.lstsq(A, y_test, rcond=None)[0]
print(f"SS lstsq: {np.sum((y_test - A@c_l)**2):.4e} <- mas estable")
```

XV. CONCLUSIONES

Se resolvieron los 14 ejercicios (67–80) del capítulo de ajuste de curvas por mínimos cuadrados. Las implementaciones en Python con NumPy confirmaron los resultados analíticos en todos los casos:

- La recta de mínimos cuadrados siempre pasa por el centroide $(\bar{X}, \bar{Y})$ (Ej. 67).
- El ajuste cambia según qué variable se considera libre de errores (Ej. 68).
- La forma $A^T A c = A^T y$ unifica todos los problemas de mínimos cuadrados (Ej. 74).
- Los modelos no lineales se linealizan con logaritmos para aplicar mínimos cuadrados directamente (Ejs. 71, 75).
- El número de condición de $A^T A$ crece exponencialmente con el grado; `numpy.linalg.lstsq` es siempre preferible a resolver las ecuaciones normales directamente (Ej. 80).